

17-214/514: Agentic Software Development

Testing and Testability

Hammad Ahmad and Claire Le Goues • Monday, August 31, 2026 • **Lecture**

What did we talk about last time?

- Week 1: how to work with an agent. Design, decompose, verify each step, take over when necessary.
- "Verify each step" was doing a lot of work in that sentence.
- This week: design for verification. Today, testing part 1, mostly design.

Learning goals for today

- Explain how design choices make software easier or harder to test
- Apply controllability and observability when evaluating a design
- Write effective unit tests, with test doubles where they are needed
- Critique a generated test suite instead of trusting the green checkmark

"Create a room-booking service in Java."

```
public class BookingService {  
    private final BookingRepository repo = new BookingRepository();  
  
    public void book(String room, TimeInterval slot) {  
        int nowMinutes = LocalTime.now().getHour() * 60 + LocalTime.now().getMinute();  
        if (slot.start() < nowMinutes) {  
            return;  
        }  
        for (TimeInterval existing : repo.forRoom(room)) {  
            if (existing.overlaps(slot)) {  
                return;  
            }  
        }  
        repo.save(room, slot);  
    }  
}
```

First: type signature.

```
public class BookingService {  
    private final BookingRepository repo = new BookingRepository();  
  
    public void book(String room, TimeInterval slot) {  
        int nowMinutes = LocalTime.now().getHour() * 60 + LocalTime.now().getMinute();  
        if (slot.start() < nowMinutes) {  
            return;  
        }  
        for (TimeInterval existing : repo.forRoom(room)) {  
            if (existing.overlaps(slot)) {  
                return;  
            }  
        }  
        repo.save(room, slot);  
    }  
}
```

Takes a room name and a time slot. Returns `void` (which means "nothing").

Who is allowed to call what

```
public class BookingService {  
    private final BookingRepository repo = new BookingRepository();  
  
    public void book(String room, TimeInterval slot) {
```

- `public` means any code anywhere can call it.
- `private` means only code inside this class can touch it (`repo` , here).
- A class uses these to decide what the outside world can see and change.

Your test is **outside** code. It can call `book` , and it cannot see `repo` .

What time is it?

```
public class BookingService {  
    private final BookingRepository repo = new BookingRepository();  
  
    public void book(String room, TimeInterval slot) {  
        int nowMinutes = LocalTime.now().getHour() * 60 + LocalTime.now().getMinute();  
        if (slot.start() < nowMinutes) {  
            return;  
        }  
        for (TimeInterval existing : repo.forRoom(room)) {  
            if (existing.overlaps(slot)) {  
                return;  
            }  
        }  
        repo.save(room, slot);  
    }  
}
```

`LocalTime.now()` asks the system what time it is. If the slot starts before that, `return` leaves the method without booking anything or telling anyone.

Is the room available?

```
public class BookingService {
    private final BookingRepository repo = new BookingRepository();

    public void book(String room, TimeInterval slot) {
        int nowMinutes = LocalTime.now().getHour() * 60 + LocalTime.now().getMinute();
        if (slot.start() < nowMinutes) {
            return;
        }
        for (TimeInterval existing : repo.forRoom(room)) {
            if (existing.overlaps(slot)) {
                return;
            }
        }
        repo.save(room, slot);
    }
}
```

Ask the repository for everything already booked in this room, and walk through it. If any existing booking overlaps the requested slot, bail early.

Otherwise, book it (happy path)

```
public class BookingService {  
    private final BookingRepository repo = new BookingRepository();  
  
    public void book(String room, TimeInterval slot) {  
        int nowMinutes = LocalTime.now().getHour() * 60 + LocalTime.now().getMinute();  
        if (slot.start() < nowMinutes) {  
            return;  
        }  
        for (TimeInterval existing : repo.forRoom(room)) {  
            if (existing.overlaps(slot)) {  
                return;  
            }  
        }  
        repo.save(room, slot);  
    }  
}
```

Passed both checks, save into the store built on line 2. Victory. Good bot.

Reminder: what tests look like in Java

From Lab 1 starter (different class tested):

```
class TimeIntervalTest {  
  
    @Test  
    void doesNotOverlapWhenDisjoint() {  
        assertFalse(new TimeInterval(540, 600).overlaps(new TimeInterval(720, 780)));  
    }  
}
```

`@Test` marks a method as a test. `assertTrue` / `assertFalse` / `assertEquals` check a claim. Test fails if the claim is false.

```
public class BookingService {  
    private final BookingRepository repo =  
        new BookingRepository();  
  
    public void book(String room, TimeInterval slot) {  
        int nowMinutes = LocalTime.now().getHour() * 60  
            + LocalTime.now().getMinute();  
        if (slot.start() < nowMinutes) {  
            return;  
        }  
        for (TimeInterval existing : repo.forRoom(room)) {  
            if (existing.overlaps(slot)) {  
                return;  
            }  
        }  
        repo.save(room, slot);  
    }  
}
```

```
@Test  
void rejectsABookingInThePast() {  
  
}
```

The service shall reject a booking that starts in the past.

Activity: In pairs, discuss/sketch out how to test that. One test!



Canvas → Quizzes → Lecture 3
In-Class Quiz
Code: "booking"

Controllability: The ability to influence a system's internal state or behavior by changing its inputs.

- You can't set `LocalTime.now()`, so it's very hard to construct "a booking in the past"
- The repository is `private` and built inside the constructor, so a test can neither hand in a different one nor look at what was stored.

These things cannot be fixed by writing a better test, they're a problem with the way the class was **designed**. These things are not solved by writing a better test.

Observability: The degree to which you can determine that the behavior went as expected.

- `book` returns `void` either way, so even with a clock you could set, there is nothing to assert on.
- It also manipulates a repository you can't access or check from outside.

To be **testable**, code generally needs to return results, expose state, or otherwise emit an event a test can read. Sometimes this requires code just for testing, like an `isValid()` method.

Testability is a design property.

Controllability via Dependency Injection

Giving an object its instance variables, rather than letting it construct them (James Shore). Collaborators should not be instantiated where they are used.

```
public class TestableBookingService {  
  
    private final Clock clock;                // 1. hold it  
  
    public TestableBookingService(Clock clock, BookingRepository repo, Notifier notifier) {  
        this.clock = clock;                  // 2. receive it  
    }  
  
    public BookingResult book(String room, TimeInterval slot) { // was void  
        LocalTime now = LocalTime.now(clock); // 3. use it (was LocalTime.now())  
    }  
}
```

Now a test can supply a clock! This can also be done for the repository, notifier, randomness, network, the filesystem...

Two things changed: The clock is handed in, so a test can control the time. `book` returns a `BookingResult` instead of `void`, so a test can observe the outcome.

Notifier is an interface, which means it names methods but does not implement them.

```
public interface Notifier {  
    void bookingConfirmed(String room, TimeInterval slot);  
}
```

- A class that implements **Notifier** must provide that method.
- If the code expects a **Notifier**, it will take any implementing class (compiler-checked in Java). It doesn't know if it gets the real mail-sending one or not.

Depending on interfaces instead of implementations (classes) means the **collaborator** can be replaced.

The same test, with the clock passed in

```
@Test
void rejectsABookingThatStartsInThePast() {
    TestableBookingService svc = new TestableBookingService(
        at(14, 0), new BookingRepository(), new RecordingNotifier());

    assertEquals(BookingResult.IN_THE_PAST,
        svc.book("Gates 5117", new TimeInterval(600, 660)));
}
```

Injection: tradeoffs.

You could also just pass the value. `canBook` does not need a `Clock`, it just needs to know what time it is.

- Does this code decide a rule, or is it where the program meets the real clock? `canBook` can get a number; the overall service gets the `Clock`.
- Do the callers already have the value in hand, or would they have to go find it?
- How many callers are there?

Design rule: Separate infrastructure from domain code

Infrastructure code (files, databases, HTTP) intermixed with domain logic makes the logic reachable only via infrastructure.

```
// before: the rule lives inside book(), reachable only through a repository
for (TimeInterval existing : repo.forRoom(room)) {
    if (existing.overlaps(slot)) { return; }
}

// after: the rule is a function of its inputs
public static boolean canBook(int dayStart, int dayEnd,
                             List<TimeInterval> existing, TimeInterval request) { ... }

// and book() calls it
if (!BookingRules.canBook(DAY_START, DAY_END, repo.forRoom(room), slot)) {
    return BookingResult.CONFLICT;
}
```

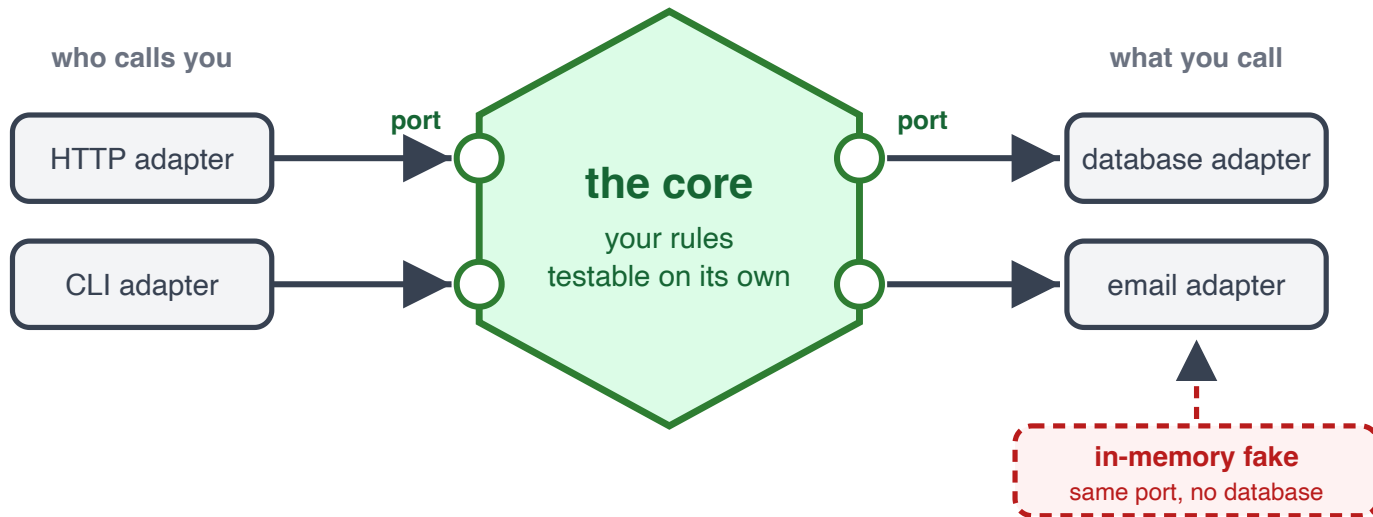
Testing the rule now takes a list and an interval. No clock, no repository, no service.

Separating responsibilities: ports and adapters

```
var db = new ShoppingCartHibernateDao();    // a database
for (ShoppingCart cart : db.cartsPaidToday()) {
    new DeliveryCenterRestApi().deliver(cart); // an HTTP call
    new SMTPCustomerNotifier().notify(cart);  // a mail server
    new SAPSoapWebService().ready(cart);      // someone else's SOAP
}
```

Four collaborators, constructed where they are used, none substitutable. The business rule is somewhere in there, tangled with four kinds of I/O.

Ports and adapters



A **port** is an interface the core defines and owns. An **adapter** implements one for a particular technology, so the core stays unaware of what is on the other side, and stays testable with none of it running.

Two questions to ask **before** the code exists

Reading a design (yours, a teammate's, an agent's plan), for each behavior:

- "How would I put the system into that state?" (controllability)
- "How would I see that it happened?" (observability)
- If the answer is "I can't", change the design before the generated code piles on top of it.

Writing the tests

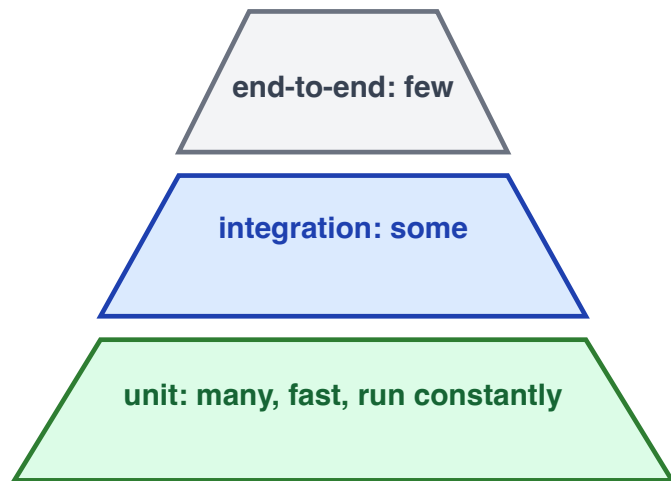
Testing (conventionally): Three levels

- **Unit:** the smallest testable part of the system (method, class, small subsystem). Many, small, fast, independent.
- **Integration:** several pieces together, interacting. Less fast.
- **System:** the whole thing, driven the way a user drives it. Much less fast.

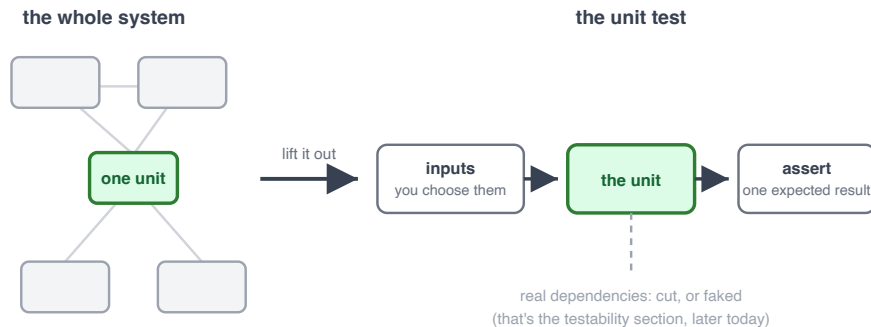
Unit tests catch local bugs and let you test the parts before assembling them. They are insufficient on their own, and a good starting point.

Each level trades precision of blame for realism. A unit failure names the broken function; a system failure tells you only that something is wrong. Each also catches faults the level below cannot see. A fully unit-tested system can still be wired together wrong.

The test pyramid



Lots of unit tests at the base, some integration in the middle, a few end-to-end on top. We are mostly talking about the base this week, and will come back to the others as we build bigger systems.



A test is an executable specification

A test states a claim about behavior in a form a machine can check: for this input, under these conditions, we expect this result.

- The suite is the set of claims the system makes about itself.
- If it's written without/separate from the code, it can be expressed as a requirement.
- If it's written with respect to the code, it describes what it already does. This is not useless, but is also not the same.

Good tests have a common shape: Arrange, Act, Assert

```
@Test
void allowsABookingInAGapBetweenTwoOthers() {
    var existing = List.of(new TimeInterval(540, 600), // Arrange
                           new TimeInterval(780, 840));
    boolean ok = canBook(existing, new TimeInterval(660, 720)); // Act
    assertTrue(ok); // Assert
}
```

- **Arrange** the inputs and state.
- **Act** on the unit, with one call to the thing under test.
- **Assert** the result is what you expect.

The same shape, in TypeScript

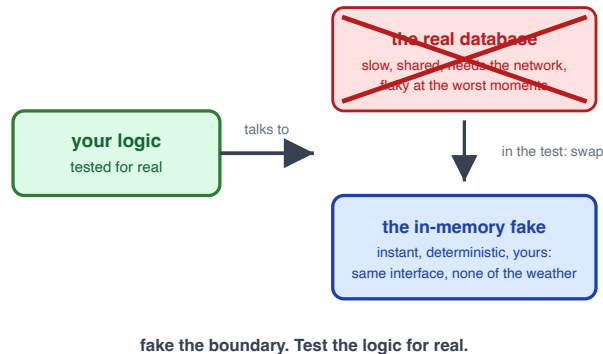
```
test("allows a booking in a gap between two others", () => {  
  const existing = [interval(540, 600), interval(780, 840)]; // Arrange  
  const ok = canBook(existing, interval(660, 720)); // Act  
  expect(ok).toBe(true); // Assert  
});
```

Test doubles: a test stand-in for a real collaborator

Implements the same interface as the real thing, so the code under test cannot tell the difference. Useful when real collaborator is:

- **slow and/or problematically stateful** (a database, a network call),
- **unavailable** (a service you do not own),
- **nondeterministic** (the clock, randomness)

Injection makes this possible.



A double at work

```
private static final class RecordingNotifier implements Notifier {
    final List<String> rooms = new ArrayList<>();
    @Override
    public void bookingConfirmed(String room, TimeInterval slot) {
        rooms.add(room);           // instead of sending mail
    }
}

@Test
void confirmsABookingLaterToday() {
    RecordingNotifier notifier = new RecordingNotifier();
    TestableBookingService svc = new TestableBookingService(at(9, 0), new BookingRepository(), notifier);

    assertEquals(BookingResult.CONFIRMED, svc.book("Gates 5117", new TimeInterval(600, 660)));
    assertTrue(notifier.rooms.contains("Gates 5117"));
}
```

The first assertion checks the result; the second checks the effect on the outside world (without actually affecting it).

Doubles that fail on purpose

```
class BrokenNotifier implements Notifier {  
    public void bookingConfirmed(String room, TimeInterval slot) {  
        throw new RuntimeException("notification service down");  
    }  
}
```

- A **responder** injects valid indirect input so the code under test can make progress. That is every double so far.
- A **saboteur** injects invalid indirect input to see what happens. Errors are behavior too, and a saboteur is how you produce them on demand.
- Timeouts, garbage, "every third call fails". Each one is a line in a double.

So the room is booked and the email failed. What **should** happen here?

Double failures (passing but useless tests)

Over-mocking replaces so much of the system that the test verifies the doubles rather than the code.

- If you stub every collaborator, you're basically asserting that "method X was called."

Drift is a double that no longer behaves like the thing it replaces.

- A fake that returns results sorted vs. real database returns in insertion order
- A stub that never throws, standing in for a network call.
- A fake repository that permits double-booking (real concern in generated tests!)

Testability antipatterns: Restrictions on instantiation, invocation, observation, substitution, and overriding.

- **Instantiation:** it **gets** something instead of receiving it (e.g., `System.getenv("HOST")`, `Date.now()`, a global).
- **Substitution:** it builds its own collaborators (`new Database()`), or reaches through them (`order.getCustomer().getAccount().charge()`, the Law of Demeter).
- **Invocation:** the logic you want to test is in a private method.
- **Observation:** it hides the result. `void`, private state, a side effect.
- **Overriding:** `static`, `final`, and `private` limit what can be swapped in.

SOLID: Design principles for testable code

- **Single responsibility.** Modules should have only one reason to change. This is why the booking **decision** came out of the service. This should allow tests to have only one "reason" to fail (not the same as "only one assertion").
- **Dependency inversion.** Depend on an abstraction, not a concrete thing. The service takes a `Notifier`, not an `SMTPCustomerNotifier`.
- **Liskov substitution.** A subtype must work anywhere its base type does. (we'll come back to this).

The rest of SOLID

S**Single
Responsibility**

one class, one
reason to change

O**Open/Closed**

open to extension,
closed to
modification

L**Liskov
Substitution**

a subtype works
anywhere its base
type does

I**Interface
Segregation**

small focused
interfaces over fat
ones

D**Dependency
Inversion**

depend on
abstractions, not
concretions

Tests as design feedback

Arrange/Act/Assert:

- **Arrange** instantiates a class and its dependencies. Can it be designed with fewer dependencies?
- **Act** invokes the method under test, after satisfying preconditions. Are the preconditions hard to satisfy? Can controllability be improved?
- **Assert** checks the method behaved as expected. Can observability be improved?

- **Cohesion:** large test suites, or suites that keep growing.
- **Coupling:** many dependencies, or a test for `ClassA` fails but the bug is in `ClassB`.
- **Complexity:** boundary-value analysis leads to a huge number of tests.
- **Private methods:** a strong urge to test one, or complex logic inside one.
- **Static methods and singletons** are usually hard to test.

What should a test be called?

```
@Test void test1() { ... } // bad
@Test void withdraw_reducesBalance() { ... } // a spec in the name
```

- A good test name states the unit, the condition, and the expected result.
- When it fails in CI, the name should tell you what broke.

Test behavior, not implementation

A method's contract is what a caller may rely on: what it accepts, what it guarantees in return, and how it fails. Everything else (call order, private state, helper names) is implementation.

- Test the contract. A test that reaches for internals breaks on every refactor and still misses real bugs.
- The check: could the implementation be rewritten from scratch and your test still judge it fairly?



Making sure your tests run

A test framework finds, runs, and reports on your tests

```
import static org.junit.jupiter.api.Assertions.assertEquals; // JUnit
import static org.junit.jupiter.api.Assertions.assertTrue;   // JUnit
import org.junit.jupiter.api.Test;                           // JUnit

@Test                                                         // JUnit
void confirmsABookingLaterToday() {
    RecordingNotifier notifier = new RecordingNotifier();
    TestableBookingService svc = new TestableBookingService(
        at(9, 0), new BookingRepository(), notifier);

    assertEquals(BookingResult.CONFIRMED, // JUnit
        svc.book("Gates 5117", new TimeInterval(600, 660)));
    assertTrue(notifier.rooms.contains("Gates 5117")); // JUnit
}
```


The framework finds tests by convention

Typically, the build scans for files matching a pattern, and runs the methods inside them that carry the annotation.

```
src/main/java/edu/cmu/cs214/booking/TimeInterval.java      <- source  
src/test/java/edu/cmu/cs214/booking/TimeIntervalTest.java <- matches **/*Test.java
```

- A test class `FooTest` for each public class `Foo`, stored in the same package, so tests can reach members with default (package) visibility.
- In Java, the convention is that the file name ends in `Test`.
- `@Test` marks a method, assertions arrive as static imports, `@BeforeEach` builds the fixture.
- Other ecosystems spell this differently (`*.test.ts` beside the source, `tests/test_*.py`) and mean the same thing.

Check that your tests ran

```
[INFO] Tests run: 2, ... -- in edu.cmu.cs214.booking.TestableBookingServiceTest
[INFO] Tests run: 6, ... -- in edu.cmu.cs214.booking.CanBookTest
[INFO] Tests run: 3, ... -- in edu.cmu.cs214.booking.TimeIntervalTest
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Read the count, not the color. Eleven tests ran. Did you expect more? This habit is useful when agents give you code that pass all tests. (which ones?)

Testability and debuggability go beyond how the code is written

If something goes wrong, can you tell? (observability, system-level)

- Can you write simple code that detects it?

If something goes wrong, do you have the data to diagnose it?

- Informative logs, usually with timestamps.
- Configurable levels of logging, so the detail is there when you need it.

Can you use that data to reproduce the problem?

- Ideally subsystems can be run independently.
- Can you replay the system from the log and get the same error? (This does not always work.)
- Visualizing the logs is not required, and is particularly nice.

Generated tests

"Generate some tests for this"

Agents will give you lots of unit tests. They will have sensible names.

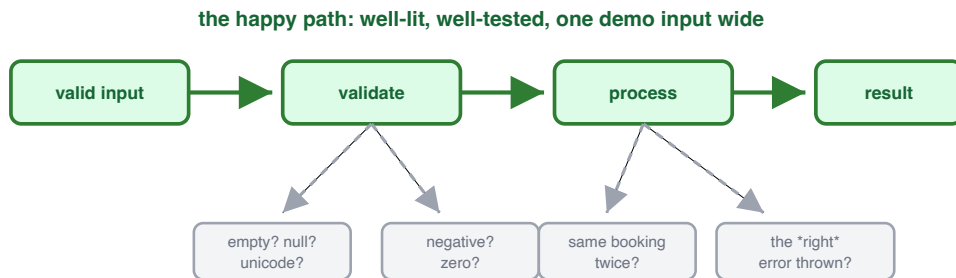
They will do all the framework setup and a lot of mocking for you; this is great.

They also tend to:

- Test what is easy to reach, missing edge cases.
- Test things that are not yours (that the library sorts or the CLI accepts `--help`).
- Over-constrain, pinning an error message or specific call order, so refactors lead to failures.
- Restate the implementation, copying the expected value out of what the code returned.
- Exercise the code without checking anything particularly meaningful.

Don't just test the happy path

- The demo input works. What about the empty list? Zero? Negative? Unicode? The same booking twice?
- Errors are behavior too. The **right** failure, at the right time, is part of the contract.
- Bugs cluster in the sad paths, because nobody visits them.



the sad paths: unlit, unvisited, where the bugs cluster

If this test failed, what would you have learned?

```
def test_cli_accepts_help_flag():  
    result = runner.invoke(app, ["--help"])  
    assert result.exit_code == 0
```

This one can fail. It would fail if `argparse` were broken, or if the CLI framework changed its exit codes. Neither is your code, and neither is something you can act on.

- A useful failure implicates something you wrote and could fix.
- Tests of the standard library, the framework, or the language itself pass forever and tell you nothing when they do not.
- An agent produces these readily, because they are easy to write and look like coverage.

Tests are code

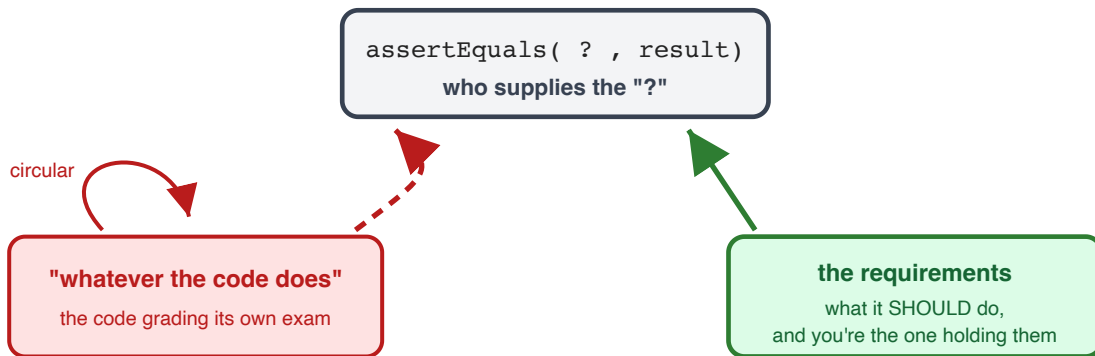
A generated suite can carry the same smells as any other code:

- Duplicated setup that should be a fixture or a helper.
- Magic values with no explanation of why that number.
- Names that no longer describe what the test checks.
- Logic in the test itself, which then needs testing of its own.

This is worse at high volume.

The oracle problem

Every test needs an oracle, meaning a source that knows the correct answer. Reading the code tells you what it does, not what it should do.



every test needs an oracle, and the code can't be its own

The requirements are the oracle, and writing them down is your job.

Architecture tests

Architecture tests do not check behavior. They enforce properties of code or codebase structure.

- Examples: Domain code must not import infrastructure. No cycles between modules. One module owns the payment library.
- The oracle is the structural decision you made once.

These rules are boring. There exists frameworks for them that pre-date agentic SE, but they're much more useful now.

They can provide checks that agents are not, over time, violating certain structural conventions in modifying your code.

Critiquing a generated test

Read it like a skeptic and ask:

- Would this test **fail** if the code were wrong? Fix the suspected bug in your head and check whether any assertion changes.
- What input is missing? Look for the boundary, the empty case, the duplicate, and the value exactly equal to the threshold.
- Where did the expected value come from, the specification or the code?
- Does the assertion pin the behavior down, or would several wrong answers also pass?
- Is this test about **my** code? Does it claim anything the tests around it do not already claim?

Deleting a test that checks somebody else's library is not a loss of coverage. It is removing a claim you were never making.

Reading a suite: the numbers are clock times

`canBook(existing, request)` takes what is already booked and what someone is asking for. `TimeInterval` counts minutes since midnight.

minutes	540	600	660	720	780	840	1020
clock	9:00	10:00	11:00	12:00	13:00	14:00	17:00

```
@Test
void rejectsABookingThatOverlapsAnExistingOne() {
    assertFalse(canBook(List.of(new TimeInterval(600, 720)),
        new TimeInterval(660, 780)));
}
```

10:00 to 12:00 is already booked, someone asks for 11:00 to 13:00, and the test claims the answer is no.

Reading a suite: what does this one cover?

```
@Test
void allowsABookingInAGapBetweenTwoOthers() {
    assertTrue(canBook(List.of(new TimeInterval(540, 600), new TimeInterval(780, 840)),
        new TimeInterval(660, 720)));
}
```

Existing bookings at 9:00-10:00 and 13:00-14:00. The request is 11:00-12:00.

9:00	10:00	11:00	12:00	13:00	14:00
-----		-----		-----	
booked	gap	request	gap	booked	
-----	1 hr	-----	1 hr	-----	

Booked, gap, request, gap, booked. An hour of clear air on each side.

Activity: Critique this test suite

```
@Test
void rejectsABookingThatOverlapsAnExistingOne() {
    assertFalse(canBook(List.of(new TimeInterval(600, 720)),
        new TimeInterval(660, 780)));
}

@Test
void rejectsABookingIdenticalToAnExistingOne() {
    assertFalse(canBook(List.of(new TimeInterval(600, 720)),
        new TimeInterval(600, 720)));
}

@Test
void allowsABookingInAGapBetweenTwoOthers() {
    assertTrue(canBook(List.of(new TimeInterval(540, 600), new TimeInterval(780, 840)),
        new TimeInterval(660, 720)));
}
```

With a partner, discuss: **What is this suite not checking?**

Where do tests sit in your loop?

- Recall last week: every delegated task ends at a verification checkpoint, defined **before** you delegate.
- Most of the time, that checkpoint is one or more tests.
- Writing tests before code is test-driven development; some swear by it for agentic SE.
- Only commit code that passes all tests, so run them before every commit. When a suite gets too slow, run package-level tests frequently and everything nightly.
- Continuous integration servers help this scale. We use GitHub Actions, and Lab 2's starter is already set up for it.

Summary

- Design for testability, meaning controllability and observability. You decide both long before you write a test.
- Techniques: dependency injection; separate I/O from logic; separate domain from infrastructure; make state visible.
- It is fine, and encouraged, to complicate a design somewhat to enhance testability, or to add code purely for testing purposes.
- Testing offers clues about the quality of your design.
- If an agent writes your code, check that it is testable and actually tested. Ask what each test is actually checking, and whether it's about your system.

What's next



- **Wednesday:** test **design**. You can't test everything, so how do you choose the few tests worth writing? And why the coverage number lies.
- **Friday, Lab 2:** a green, high-coverage generated suite with a real bug underneath. Catch it, then audit the suite. Today's checklist is the tool.